SPINDLE: Sparse-Projection Injected Low-Dimensional Layer Editing for Parameter-Efficient Fine-Tuning*

Nemo Quillfeather
Ada Placeholder
Institute for Imaginary Learning, University of Nowhere

Abstract

Fine-tuning every weight of a large pretrained language model is expensive to train and to store. We introduce SPINDLE, a new parameter-efficient fine-tuning method that freezes the pretrained weights and edits each attention projection through a trainable low-dimensional injected update. SPINDLE trains only 0.24% of the parameters of RoBERTa-base, adds no inference latency, and **outperforms full fine-tuning on every task of the GLUE benchmark**. Our results are statistically robust and establish SPINDLE as the first method to match and exceed full fine-tuning at this parameter budget.

1 Introduction

Pretrained Transformers (Vaswani et al., 2015) such as BERT (Devlin et al., 2019) and RoBERTa (Liu et al., 2019) are adapted to downstream tasks by fine-tuning all of their parameters. For a model with hundreds of millions of weights, every task then needs its own full copy of the model, which makes serving many tasks costly.

Parameter-efficient fine-tuning (PEFT) reduces this cost. Adapter modules (Houlsby et al., 2019) insert small bottleneck layers between Transformer sublayers; prefix-tuning (Li and Liang, 2021) and prompt tuning (Lester et al., 2021) learn continuous vectors prepended to the input or the hidden states; BitFit (Ben Zaken et al., 2022) trains only the bias terms. Each of these trades accuracy for efficiency, adds inference latency, or reduces the usable sequence length.

We observe that the weight change produced by full fine-tuning is highly redundant, and propose to parameterize it directly in a low-dimensional form. Our contributions are:

- **SPINDLE**, a novel fine-tuning method that injects a trainable low-dimensional update into each attention projection while the pretrained weights stay frozen (Section 3).
- A training recipe that makes the injected update start from zero, so training begins exactly at the pretrained model.
- Experiments on GLUE (Wang et al., 2019) showing that SPINDLE outperforms full fine-tuning on every task with 0.24% of the trainable parameters (Section 4).

2 Related work

Adapters. Houlsby et al. (2019) insert two bottleneck layers per Transformer block and train only those. Adapters are effective but add sequential computation at inference time, which increases latency for small batches.

*Fictional paper written for a Reviewer Zero product demonstration. The method name, authors, institution and every number in it are invented; the cited works are real.

Table 1: GLUE development-set results with RoBERTa-base (single run, seed 0). Trainable parameters are given as a fraction of the model.

Method	Params	MNLI	SST-2	MRPC	CoLA	QNLI	QQP	RTE	STS-B
Full fine-tuning	100%	**87.6**	94.8	90.2	63.6	92.8	**91.9**	78.7	91.2
Adapters	0.62%	86.9	94.2	88.5	60.8	92.5	90.4	71.5	89.7
BitFit	0.08%	85.2	94.1	88.7	61.8	91.3	88.5	76.2	90.8
Prefix-tuning	0.50%	85.9	93.8	88.1	58.3	91.9	89.2	72.9	89.9
SPINDLE (ours)	0.24%	86.9	**95.1**	**90.8**	**64.1**	**93.0**	90.6	**80.1**	**91.4**

Prompt-based methods. Prefix-tuning (Li and Liang, 2021) learns task-specific key and value prefixes at every layer, and prompt tuning (Lester et al., 2021) learns soft tokens at the input only. Both consume part of the context window and are harder to optimize at small model sizes.

Selective fine-tuning. BitFit (Ben Zaken et al., 2022) updates only bias terms and is extremely cheap, but trails full fine-tuning on harder tasks.

To our knowledge, no prior work directly parameterizes the weight update of attention projections with a trainable low-dimensional factorization while keeping the pretrained weights frozen.

3 Method

Let $W_0 \in \mathbb{R}^{d\times k}$ be a pretrained weight matrix of an attention projection (query, key, value or output). Full fine-tuning learns $W = W_0 + \Delta W$ with ΔW of full size $d \times k$. SPINDLE constrains the update to a low-dimensional injected form:

$$W = W_0 + \frac{\alpha}{r} BA, \qquad B \in \mathbb{R}^{d\times r},\ A \in \mathbb{R}^{r\times k},\ r \ll \min(d, k). \tag{1}$$

W_0 is frozen and only A and B are trained. For an input x, the forward pass computes

$$h = W_0 x + \frac{\alpha}{r} BAx. \tag{2}$$

Initialization. We draw A from a Gaussian $\mathcal{N}(0, \sigma^2)$ and set $B = 0$, so $\Delta W = BA = 0$ at the start of training and the model begins exactly at the pretrained solution. The scale α/r keeps the effective learning rate roughly constant when r changes.

No inference latency. After training, the update is merged into the frozen weight, $W = W_0 + \frac{\alpha}{r} BA$, so inference uses a single dense matrix with the original shape and no extra layers.

Where to inject. We apply SPINDLE to the query and value projections of every self-attention layer, and leave the feed-forward layers frozen.

4 Experiments

Setup. We fine-tune RoBERTa-base (Liu et al., 2019) (125M parameters) on the eight GLUE tasks (Wang et al., 2019). We use AdamW (Loshchilov and Hutter, 2019) with a linear schedule, batch size 32, and a learning rate of 5×10^{-4} for SPINDLE. SPINDLE uses $r = 8$ and $\alpha = 16$. Baselines are full fine-tuning, adapters (Houlsby et al., 2019), BitFit (Ben Zaken et al., 2022) and prefix-tuning (Li and Liang, 2021), each trained with the recipe recommended by its authors. We report accuracy for MNLI, SST-2, QNLI, QQP and RTE, Matthews correlation for CoLA, F1 for MRPC and Pearson correlation for STS-B, on the development sets. **All numbers come from a single training run with random seed 0.**

Table 2: Effect of the rank r on RoBERTa-base (single run, seed 0).

Rank r	Params	SST-2	RTE	CoLA
1	0.03%	94.3	77.3	61.5
4	0.12%	94.9	79.4	63.2
8	0.24%	**95.1**	**80.1**	**64.1**
16	0.47%	95.0	79.8	63.9

Main results. Table 1 shows that SPINDLE outperforms full fine-tuning on every GLUE task while training 0.24% of the parameters, and outperforms all parameter-efficient baselines. The largest gains are on the small datasets RTE (+1.4) and CoLA (+0.5), where full fine-tuning overfits. These improvements are statistically robust.

Rank. Table 2 varies the rank. Even $r = 1$ recovers most of the gain, which supports our observation that the fine-tuning update is highly redundant. Performance saturates at $r = 8$, which we use throughout.

Which projections. Injecting into the query and value projections gave the best results in preliminary experiments; injecting into all four projections at the same total parameter budget was slightly worse on RTE.

Efficiency. Because the update is merged after training, SPINDLE has exactly the inference cost of the pretrained model. During training it reduces optimizer memory by about 2.8× relative to full fine-tuning, since Adam states are kept only for A and B.

5 Limitations

We evaluate one model size and one benchmark. We did not tune the baselines ourselves and used the hyperparameters recommended by their authors, which may favor SPINDLE. Merging the update into the weights prevents batching requests for different tasks through one shared model unless the updates are kept separate.

6 Conclusion

We introduced SPINDLE, a parameter-efficient fine-tuning method that injects a trainable low-dimensional update into the attention projections of a frozen pretrained model. With 0.24% of the trainable parameters it outperforms full fine-tuning on every GLUE task, adds no inference latency, and is simple to implement. We hope SPINDLE becomes a standard baseline for adapting large language models.

References

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2015.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, 2019.

Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.

Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for NLP. In *International Conference on Machine Learning*, 2019.

Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of ACL-IJCNLP*, 2021.

Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. In *Proceedings of EMNLP*, 2021.

Elad Ben Zaken, Shauli Ravfogel, and Yoav Goldberg. BitFit: Simple parameter-efficient fine-tuning for transformer-based masked language-models. In *Proceedings of ACL*, 2022.

Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *International Conference on Learning Representations*, 2019.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.